

SKILL – S3 & S4

SESSION-3

TASK 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
my_shell> ^X^C
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ nano history.c
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ gcc history.c -o history
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ ./history
my_shell> hello
You entered: hello
my_shell> world
You entered: world
my_shell> test
You entered: test
my_shell> hello
You entered: hello
my_shell> test
You entered: test
my_shell> world
You entered: world
my_shell> abcde
You entered: abcde
my_shell> exit
Exiting shell...
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ |
```

TASK 2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ nano dynamic_memory.c
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ gcc dynamic_memory.c -o dynamic_memory
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ ./dynamic_memory
Dynamic Array:
Array resized to 10 elements.
1 2 3 4 5 6 7 8 9 10
Linked List:
3 2 1
Memory released successfully.
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ |
```

```

koumudi@Koumudi:~/OSSP/SKILL/Session_3$ valgrind --leak-check=full ./dynamic_memory
==1987== Memcheck, a memory error detector
==1987== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1987== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1987== Command: ./dynamic_memory
==1987==
Dynamic Array:
Array resized to 10 elements.
1 2 3 4 5 6 7 8 9 10
Linked List:
3 2 1
Memory released successfully.
==1987==
==1987== HEAP SUMMARY:
==1987==    in use at exit: 0 bytes in 0 blocks
==1987==   total heap usage: 6 allocs, 6 frees, 1,132 bytes allocated
==1987==
==1987== All heap blocks were freed -- no leaks are possible
==1987==
==1987== For lists of detected and suppressed errors, rerun with: -s
==1987== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ |

```

SESSION – 4

TASK 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```

koumudi@Koumudi:~/OSSP/SKILL/Session_3$ ^C
koumudi@Koumudi:~/OSSP/SKILL/Session_3$ cd ..
koumudi@Koumudi:~/OSSP/SKILL$ mkdir Session_4
koumudi@Koumudi:~/OSSP/SKILL$ cd Session_4
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ pwd
/home/koumudi/OSSP/SKILL/Session_4
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ touch tokenizer.c
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ nano tokenizer.c
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ gcc tokenizer.c -o tokenizer
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ ./tokenizer
Enter command: ls -l | grep txt

Tokens:
Token 1: ls                Type: WORD
Token 2: -l                Type: WORD
Token 3: |                 Type: DELIMITER
Token 4: grep              Type: WORD
Token 5: txt               Type: WORD
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ |

```

TASK 2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
Token 3: txt      Type: WORD
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ touch parser.c
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ nano parser.c
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ gcc parser.c -o parser
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ ./parser
Enter command: ls -l | grep txt
```

Syntax is valid.

Parse Tree / Execution Structure:

- COMMAND: ls
 - ARGUMENT: -l
- PIPE
- COMMAND: grep
 - ARGUMENT: txt

```
Dash: Syntax Error: Invalid pipe placement.
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ ./parser
Enter command: ls ||
Syntax Error: Invalid pipe placement.
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ ./parser
Enter command: | ls
Syntax Error: Invalid pipe placement.
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ ./parser
Enter command:
Error: Empty command.
koumudi@Koumudi:~/OSSP/SKILL/Session_4$ |
```